

INF221 — Lecture 9

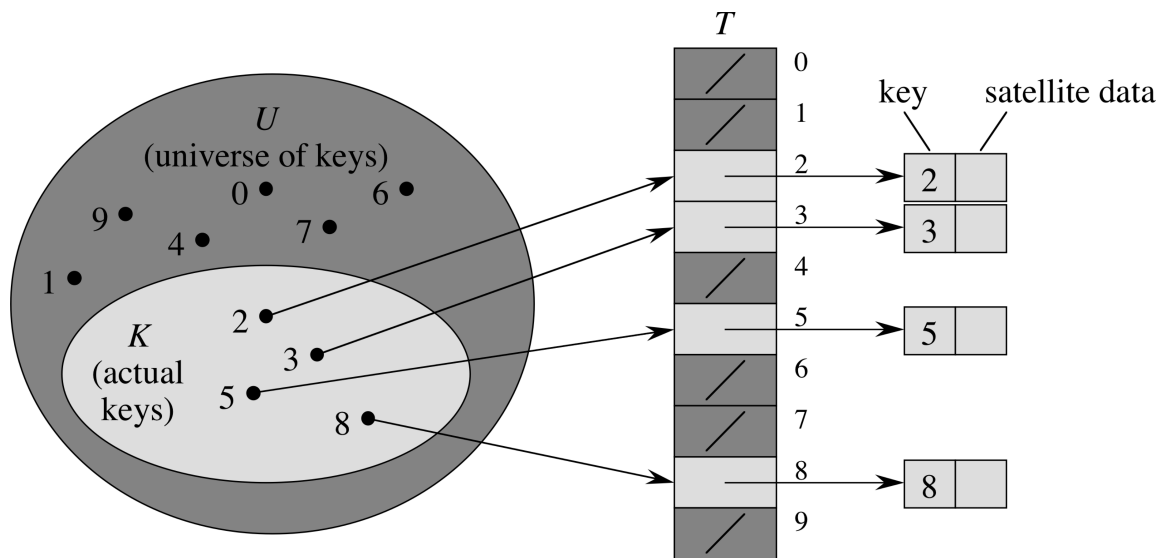
Today's topic — Hash tables

1. Direct-address tables
2. Hash tables
3. Hash functions
4. Open addressing

Hash tables—background

- Compendium parts 12 and 13
- Frequently need dynamic sets supporting dictionary operations (insert, delete, search) only
- Examples
 - dictionaries in Python
 - especially those used internally to manage names
- Problem
 - array is fast
 - but works only if we can index directly
 - set of potential names is "infinite": cannot use them directly as indices
- Solution: hash tables
 - $O(1)$ expected time for search
 - $\Theta(n)$ worst-case search time
- Hash tables generalize arrays:
 - Consider only positive integers as keys
 - Normal array
 - key k used directly as array index
 - *direct addressing*
 - only useful if we can create array with one entry per possible key value
 - Hash table
 - used when number of stored keys is small compared to number of all possible keys
 - array with size proportional to number of *stored* keys
 - for key k , use $h(k)$ as array index
 - h is a *hash function*
- Questions
 - What are good hash functions?
 - What do we do if keys k and j hash to same $h(k) = h(j)$?

Direct-address tables



(CLRS, Ch 11)

Scenario

- We need to maintain a dynamic set
- Keys are drawn from *universe* $U = \{0, 1, \dots, m - 1\}$, m not too large
- No two elements have the same key
- Satellite data may be attached to key, ignored here for simplicity

Implementation

- Implement as *direct-address table*, i.e., as array $T[0 \dots m - 1]$
- Each *slot* (position) corresponds to a key in U
- Let x be an element in the set with key k . Then $T[k]$ contains a reference (pointer) to x
- Otherwise, $T[k]$ is *NIL*

Dictionary operations

- All $O(1)$

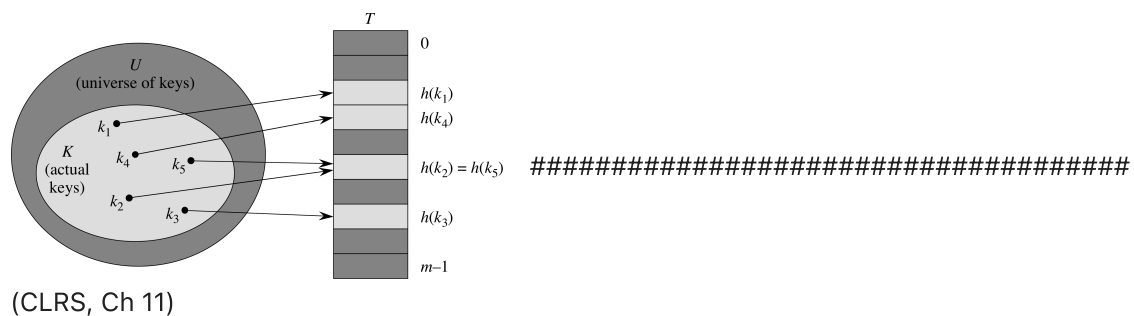
```
Direct-Address-Search(T, k)
    return T[k]
```

```
Direct-Address-Insert(T, x)
    T[x.key] = x
```

```
Direct-Address-Delete(T, x)
    T[x.key] = NIL
```

Hash tables

- Direct addressing becomes problematic if $|U|$ is large: Table T would require too much memory
- Often only a small proportion of all keys are used
- Example:
 - ca 200 billion eight-letter words (26^8) (universe)
 - languages usually have some 10^5 words (and we don't use all of them as keys)
- Let $K \subset U$ be the set of keys actually used
- Hash tables useful if $|K| \ll |U|$
- Our requirements on hash tables
 - Storage $\Theta(|K|)$
 - Average case search time $O(1)$
- See also https://en.wikipedia.org/wiki/Hash_table



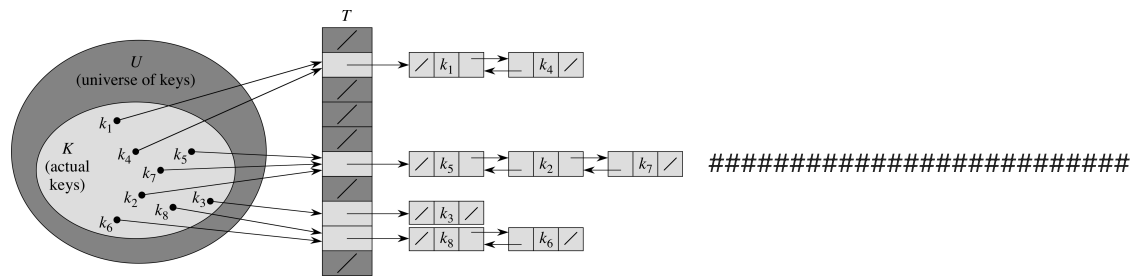
Idea

- Let $T[0 \dots m - 1]$ be a suitable large array
- Choose suitable *hash function* $h : U \rightarrow \{0, 1, \dots, m - 1\}$
- Then for any key $k \in K \subset U$, $h(k)$ is valid index to T
- k hashes to $h(k)$

Problem: Collisions

- May occur if more keys than slots, i.e., $|U| > m$
- For given set of keys K ,
 - collisions must occur if $|K| > m$
 - collisions may occur even if $|K| \leq m$
- Collision handling techniques
 - chaining
 - open addressing (below)

Chaining



(CLRS, Ch 11)

- Place all elements with same hash $h(k)$ into *doubly linked list*
- Doubly (instead of singly) linked list makes deletion faster
- Each entry of T is
 - NIL if no key in table hashes to the entry
 - reference to head of list of elements hashing to the entry

Implementation

Insert

```
Chained-Hash-Insert( $T, x$ )
    Linked-List-Insert( $T[h(x.key)]$ ,  $x$ )
```

- Worst case time is $O(1)$

Search

```
Chained-Hash-Search( $T, k$ )
    return Linked-List-Search( $T[h(k)]$ ,  $k$ )
```

- Run-time proportional to length of list for $h(k)$

Delete

```
Chained-Hash-Delete( $T, x$ )
    Linked-List-Delete( $T[h(x.key)]$ ,  $x$ )
```

- Reference to x is provided
- Therefore effort $O(1)$ in the worst case
- If we need to find the element x in list, $O(\text{length of list})$

Analysis

- Insertion and Deletion are trivial
- Need only analyse search in detail
- Assume calculation of $h(k)$ is $\Theta(1)$
- Best case
 - no collisions

- all slots either empty or single element
 - search takes $O(1)$
- Worst case
 - all k hashed to same $h(k)$
 - all entries in single slot with list length n
 - search time $O(n)$
- Consider average case below
 - How long does it take to find entry with key k ?
 - How long does it take to find that there is no entry with key k ?
- *Load factor*
 - n is number of entries in table
 - m is number of slots in table (number of linked lists, may be empty)
 - $\alpha = \frac{n}{m}$ is *load factor*
 - $\alpha < 1$: few entries, mostly short (or empty) lists
 - $\alpha > 1$: at least some collisions (lists with at least two elements)
- *Simple uniform hashing*
 - any given entry (key) is equally likely to hash into any of the m slots
 - is idealization, requires good hash function h
 - adversary could create data violating the assumption, but we will work with it here
- Expected list length under uniform hashing
 - let n_j be the length of list $T[j]$ for $j = 0, 1, \dots, m - 1$
 - thus $n = n_0 + n_1 + \dots + n_{m-1}$
 - expectation $E[n_j] = \alpha = n/m$

Unsuccessful search

Theorem

An unsuccessful search (in a hashtable with chaining and under the assumption of simple uniform hashing) takes expected time $\Theta(1 + \alpha)$.

Proof

1. Calculating the hash $h(k)$ takes time $\Theta(1)$.
2. Since we assume simple uniform hashing, any key k not in the table will be hashed to any of the m slots with equal probability. For the given key k , we need to search list $T[h(k)]$.
3. To search for key k , we need to perform a linear search through list $T[h(k)]$. Since the search is unsuccessful by assumption, we need to inspect the entire list, taking $\Theta(n_{h(k)})$ steps.
4. Because any slot is selected with equal probability, the expected length of the chain selected is $E[n_{h(k)}] = \alpha$. Thus, the linear search takes time $\Theta(\alpha)$.
5. In total we thus have shown that unsuccessful search takes $\Theta(1 + \alpha)$.

Successful search

Theorem

A successful search takes expected time $\Theta(1 + \alpha)$.

Intuitive explanation

- Unsuccessful search is worst case, since we have to search longest
- In successful search, we search on average half-way through the list that the key hashes to
- *Complication:*
 - lists have different lengths
 - since we assume key is in table, key hashes to longer lists with higher probability

Proof

We search for some element x in the table with key $k = x.key$. Assume that this element is any of the n elements in the table with equal probability.

We then search in list $T[h(k)]$ from the beginning of the list until we find x . Since we search from and insert at the head of the list, we need to check all elements that were inserted *after* x before we get to x . The number of search steps is thus then number of elements inserted after x plus one. We thus need to find the expectation for this number.

Let x_i for $i = 1, 2, \dots, n$ be the i^{th} element inserted into the table, and let $k_i = x_i.key$ be its key.

Now define the indicator variable

$$X_{ij} = I\{h(k_i) = h(k_j)\} = I\{i \text{ and } j \text{ hash to same slot}\}.$$

Further, let the element x that we search for be x_i , i.e., the element we search for was the i^{th} element inserted into the hash table. Since we assume simple uniform hashing, we have for given i

$$\Pr\{h(k_i) = h(k_j)\} = 1/m \implies E[X_{ij}] = 1/m.$$

For a given element x with insertion index i , the number of elements inserted

- into the same list as x
- *after* x is given by

$$\sum_{j=i+1}^n X_{ij}$$

- We only sum over $j > i$ to consider only entries inserted after $x = x_i$.
- The indicator X_{ij} is zero for all x_j not hashed to the same list as $x = x_i$.

We now need to average over all possible search targets and take the expectation over all possible X_{ij} :

$$\begin{aligned}
 E \left[\frac{1}{n} \sum_{i=1}^n \left(1 + \sum_{j=i+1}^n X_{ij} \right) \right] &= \frac{1}{n} \sum_{i=1}^n \left(1 + \sum_{j=i+1}^n \frac{1}{m} \right) \\
 &= 1 + \frac{1}{nm} \sum_{i=1}^n (n - i) \\
 &= 1 + \frac{n}{2m} - \frac{1}{2m} \\
 &= 1 + \frac{\alpha}{2} - \frac{\alpha}{2n} \quad \left[\text{note that } \frac{\alpha}{2} = \frac{n}{2m} \Leftrightarrow \frac{\alpha}{2n} = \right.
 \end{aligned}$$

This gives the time for the search. Adding the time to compute the hash function, which was $\Theta(1)$, we have for a successful search

$$\Theta \left(2 + \frac{\alpha}{2} - \frac{\alpha}{2n} \right) = \Theta(1 + \alpha) .$$

Interpretation

We assume that the number of slots is at least proportional to the number of elements in the table, i.e., $n = O(m)$. Thus we have

$$\alpha = \frac{n}{m} = \frac{O(m)}{m} = O(1) .$$

This means that also search takes only $O(1)$ and thus all operations are $O(1)$.

Remember that this result depends on

- a number of slots at least proportional to the number of elements
- simple uniform hashing.

Hash functions

This section does not deal with [cryptographic hash functions](#) used to fingerprint documents. INF240 has more on those.

In [34]: `! sha256sum INF221_V26_L09.ipynb`

```
0dad45d540c87c7c317e8f237607d0ed979770e78b86e4a416abdd87f39f1c8  INF221_V26_L09.ipynb
```

Good hash functions

- Should be fast

- Should provide simple uniform hashing
- Simple uniform hashing is impossible to guarantee in practice, since distribution of keys not known in advance
- Thus, use heuristics (experience) to choose hash functions that work well for the domain of keys at hand

Keys as natural numbers

- Assume that all keys are natural numbers
- Where keys are strings etc, transform them to natural numbers, e.g.

```
In [35]: def key(s):
          return sum(ord(c) * 256**i for i, c in enumerate(s))

key('a'), key('A'), key('ABC'), key('NMBU')
```

```
Out[35]: (97, 65, 4407873, 1430408526)
```

- In contrast to Exercise 3 from Assignment 1, we do not require that the numeric keys generated here have the same ordering as the strings, so we can use a simpler algorithm that works for strings of arbitrary length.

A very simple hash function

- Assume keys are uniformly distributed random numbers from $0 \leq k < 1$
- Then

$$h(k) = \lfloor km \rfloor$$

is a hash function providing simple uniform hashing

- But: assumption usually not fulfilled

Division method

$$h(k) = k \bmod m .$$

Example

$$m = 20, k = 91 \implies h(k) = 91 \bmod 20 = 11.$$

Advantage

Fast, just single division.

Disadvantage

Need to avoid certain values of m :

- Powers of 2 are bad: $m = 2^p$ for $p \in \mathbb{N}$ make $h(k)$ the p least significant bits of k , usually not random

- When converting strings as in the example above using radix 2^p (example above: $2^8 = 256$), then $m = 2^p - 1$ is bad: permuting characters does not change hash value (the proof of this is a bit challenging ...)

Good choice for m

A prime number not too close to a power of 2.

Multiplication method

$$h(k) = \lfloor m (kA - \lfloor kA \rfloor) \rfloor \quad \text{for constant } 0 < A < 1$$

- Term in parenthesis is fractional part of kA

Advantage

Value of m not critical.

Disadvantage (according to CLRS)

Slower than division method. I would claim that the proof is in the pudding: division is usually considerably slower than multiplication.

Choosing A

- All A will work, but some are better than others depending on set of keys
- Knuth suggests to use the golden ratio

$$A \approx \frac{\sqrt{5} - 1}{2} \approx 0.618$$

Example: Hashing family names

The following file contains all family names which occurred at least 200 times in the Norwegian population for at least one year between 2018 and 2024, altogether 3676 names. For each name, the proportional share among these 3676 names is given. The total number of name bearers is about 2.8 million, i.e., only about 50% of the total population. Everyone else has names occurring fewer than 200 times. The data is thus not fully representative. I obtained the data from SSB, Table 12891 (<https://www.ssb.no/statbank/table/12891>).

```
In [36]: import pandas as pd
import numpy as np
```

Load data and convert names into numbers using the `key()` function defined above.

```
In [37]: names = pd.read_csv('names.csv')
names['Key'] = names.apply(lambda r: key(r['Navn']), axis=1)
```

Assume that we want to create a table that we expect to contain about 100 entries, e.g., the member register of a small club. We want to leave some space in the table to reduce the risk of collision, so a table size of $m \approx 200$ seems sensible. We choose `m = 199`, since it is a prime number not close to a power of 2.

We calculate the hash according to the division method:

```
In [38]: names['DivHash'] = (names['Key'] % 199).astype(int)
```

And also according to the multiplication using the golden ratio. The `% 1` gives us the fractional part of the product.

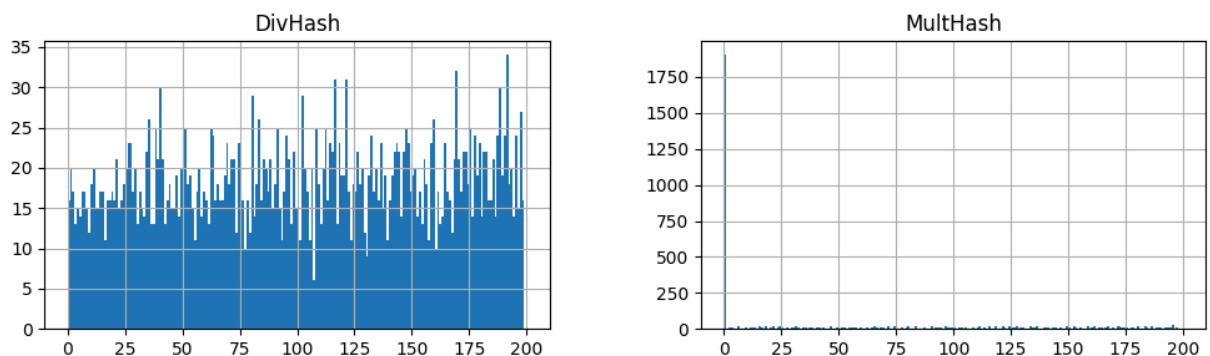
```
In [39]: A = (5*0.5 - 1) / 2
names['MultHash'] = np.floor(199 * ((names['Key'] * A) % 1)).astype(int)
```

```
In [40]: names.head()
```

```
Out[40]:
```

	Navn	Share	Key	DivHash	MultHash
0	Aa	0.000084	24897	22	38
1	Aabel	0.000087	465557414209	102	134
2	Aaberg	0.000128	113741024878913	3	125
3	Aaby	0.000094	2036490561	181	92
4	Aabø	0.000082	4167196993	81	97

```
In [41]: names.hist(['DivHash', 'MultHash'], bins=np.arange(0, 201), figsize=(12, 3))
```



- The DivHash looks quite evenly distributed
- MultHash looks very off—what is the problem?
 - Almost everyone seems to end up in bin 0
- Consider the size of the keys:

```
In [42]: max(names.Key)
```

```
Out[42]: 2239102829888182913457568508700739
```

That is a large number, more precisely

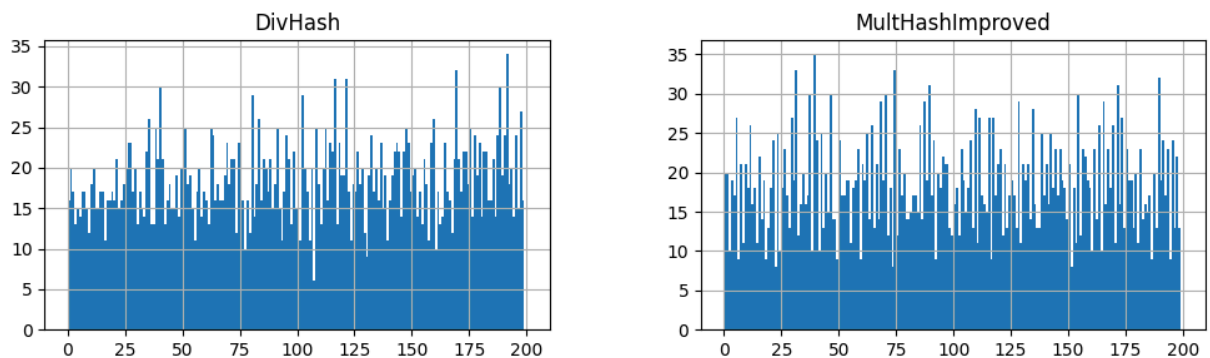
```
In [43]: float(max(names.Key))
```

```
Out[43]: 2.239102829888183e+33
```

- Floating point numbers have only about 16 decimals
- Using the arithmetic above, all numbers greater than 10^{16} will thus be mapped to zero
- We need to use Python's arbitrary precision integer arithmetic to get the fractional part
- The code below is not particularly optimal, one could likely find better solutions (but it works)
- Note that in languages such as C, C++, Java, ..., we cannot implement this hash function with built-in integer types because of their limited value range. One needs to use arbitrary precision libraries such as [GMP](#).
- Below, I approximate the golden ratio as 618/1000 and use `//` for integer division in Python.

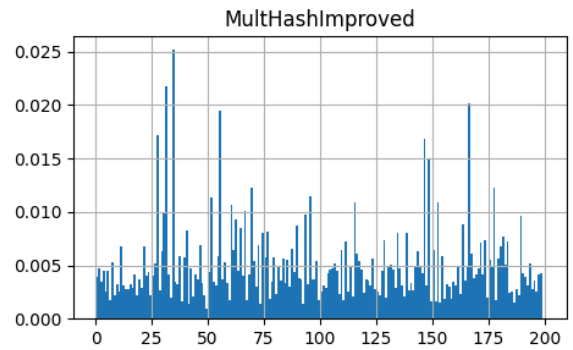
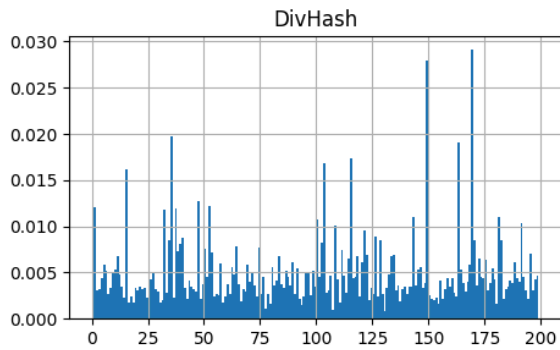
```
In [44]: nk = names['Key']  
names['MultHashImproved'] = np.floor((199 * ((nk * 618 - ((nk*618)//1000)*1000))
```

```
In [45]: names.hist(['DivHash', 'MultHashImproved'], bins=np.arange(0, 201), figsize=
```



- Now the multiplicative method looks much better.
- The histograms above ignore that different names occur with different frequency. We can include that in weighted histograms:

```
In [46]: names.hist(['DivHash', 'MultHashImproved'], weights=names.Share, bins=np.arange
```

- We clearly see that some names are more frequent than others.
- What does that mean for our search analysis above?

Open Addressing

- Compendium, Part 13
- Alternative approach to chaining
- Also known as *probing*
- Store all keys in hash table, do not attach lists
- Each slot contains exactly one key or NIL
- If a slot is occupied, try alternative slots according to a certain rule
- Searching for key k :
 1. Compute $h(k)$ and check (probe) slot $h(k)$
 2. If k in slot $h(k)$: success
 3. Otherwise, probe other slots until k found or NIL
- Sequence of slots probed must be a permutation of slot numbers $\langle 0, 1, \dots, m - 1 \rangle$
- Hash function now maps key and probe number to slot number

$$h : U \times \{0, 1, \dots, m - 1\} \rightarrow \{0, 1, \dots, m - 1\} .$$

- Thus we have *probe sequence* $\langle h(k, 0), h(k, 1), \dots, h(k, m - 1) \rangle$
- Insertion: works the same way as searching, insert into first slot that is NIL

Implementation

Searching

```

Hash-Search(T, k)
  i = 0
  repeat
    j = h(k, i)
    if T[j] == k
      return j
    i = i + 1

```

```

until T[j] == NIL or i == m
return NIL

```

Insertion

```

Hash-Insert(T, k)
  i = 0
  repeat
    j = h(k, i)
    if T[j] == NIL
      T[j] = k
      return j
    else
      i = i + 1
  until i == m
  error "table full"

```

Deletion

- Difficult
- Solution: Mark table entries as "DELETED"
 - Skip when searching
 - Insert to first "DELETED" (or NIL) entry found
- Disadvantages
 - Requires second "magic value" in addition to NIL
 - Since we must pass DELETED entries, search and insertion times no longer shrink when load factor α goes down.
- Alternative approach
 - Assume we delete key k from position j in the probe sequence, i.e., from entry $h(k, j)$.
 - Then move all entries in positions $j + 1$ and later in the same probe sequence forward.
 - Questions:
 - How many entries do we need to move?
 - How could we minimize the number of entries we need to move?

Probe sequences

- Key to successful open addressing are good probe sequences $h(k, i)$
- Ideally, they should lead to uniform hashing
- Difficult in practice, we will consider some practical approaches
- All make use of *auxiliary hash function*

$$h'(k) : U \rightarrow \{0, 1, \dots, m - 1\}$$

Linear probing

- For key k , start probe sequence at $h'(k)$
- If slot is occupied, move to next slot
- Repeat until empty slot found, wrap at end of table
- Resulting hash function is

$$h(k, i) = (h'(k) + i) \bmod m$$

- Problem
 - *Primary clustering*: long runs of occupied sequences
 - Assume a slot is preceded by j occupied slots
 - Then the probability that that slot gets filled in next insertion is $(j + 1)/m$
 - Leads to longer search and insertion times

Quadratic probing

- Similar to linear probing, but move in quadratic intervals
- Hash function

$$h(k, i) = (h'(k) + c_1 i + c_2 i^2) \bmod m \quad \text{for } c_1, c_2 > 0$$

- Need to choose c_1, c_2, m properly to achieve good hashing
- Challenge is to find numbers that will "reach" most of the table. There is a risk that some entries never get tried.
- E.g., for m prime and $c_1 = c_2 = 1/2$, only half of the entries will be tried.

Probing all elements

- Based on Weiss, Data Structures and Algorithm Analysis in C++, 3rd ed
- If table size m is prime and of form $m = 4j + 3$, then for

$$h(k, i) = (h'(k) + (-1)^i i^2) \bmod m$$

the entire table is probed.

- $(-1)^i$ means that the squares that are added have alternating signs, $-1, 4, -9, \dots$

Double hashing

- Use two auxiliary hashing functions
 - $h_1(k)$ provides starting point for probe sequence
 - $h_2(k)$ provides step length for probe sequence
- Hash function

$$h(k, i) = (h_1(k) + i h_2(k)) \bmod m$$

- If $h_2(k)$ is relatively prime to m (no common factor), this yields a probe sequence that is full permutation of $\langle 0, 1, \dots, m - 1 \rangle$
- Can be achieved by, e.g.,

- m is power of 2, $h_2(k)$ returns only odd numbers
- m is prime, $1 < h_2(k) < m$
- Yields $\Theta(m^2)$ different probe sequences: each combination of $h_1(k)$ and $h_2(k)$ gives different sequence

Performance of open addressing

Provided that

- the table never fills, so that $0 \leq n < m$ and load factor $\alpha < 1$
- uniform hashing is given
- no items are deleted from the table
- each key is equally likely to be searched for

then the expected number of probes is at most

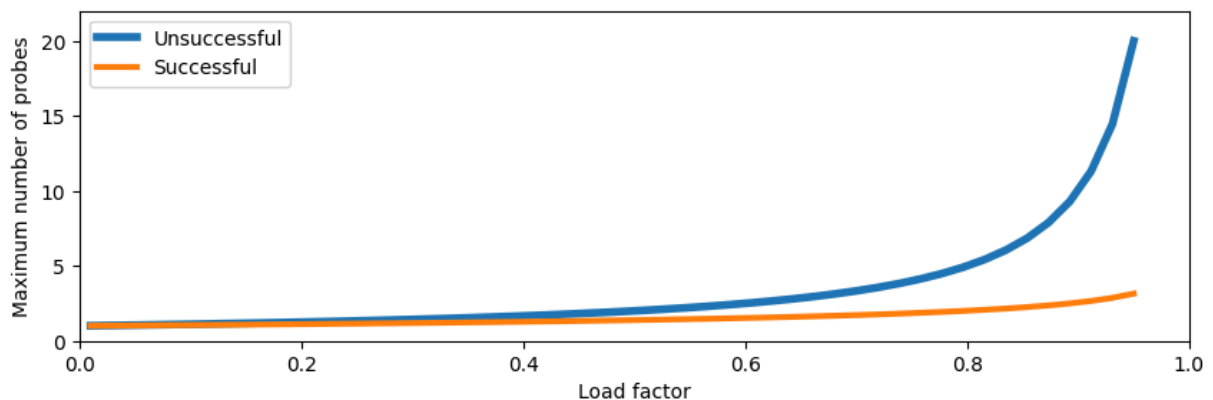
$$\frac{1}{1 - \alpha} \quad \text{in an unsuccessful search} \quad (5)$$

$$\frac{1}{\alpha} \ln \frac{1}{1 - \alpha} \quad \text{in a successful search .} \quad (6)$$

We skip the proof of this here. You can find it e.g. in CLRS.

```
In [17]: import matplotlib.pyplot as plt
import numpy as np
```

```
In [18]: alpha = np.linspace(0.01, 0.95)
plt.figure(figsize=(10, 3))
plt.plot(alpha, 1/(1-alpha), lw=4, label='Unsuccessful')
plt.plot(alpha, 1/alpha*np.log(1/(1-alpha)), lw=3, label='Successful')
plt.legend();
plt.xlabel('Load factor')
plt.ylabel('Maximum number of probes')
plt.xlim(0, 1);
plt.ylim(0, 22);
```



- Indicates general issue with open addressing: performance deteriorates significantly for load factor $\alpha \gtrsim 0.8$
- Very roughly
 - chaining more efficient for dense tables
 - open addressing more efficient for sparse tables